TECNICATURA UNIVERSITARIA
EN PROGRAMACIÓN
A DISTANCIA

TRABAJO PRÁCTICO 1 – FACTORY METHOD

Sistema de reportes con múltiples formatos

Metodología de Sistemas II – Tecnicatura Universitaria en Programación a Distancia – UTN

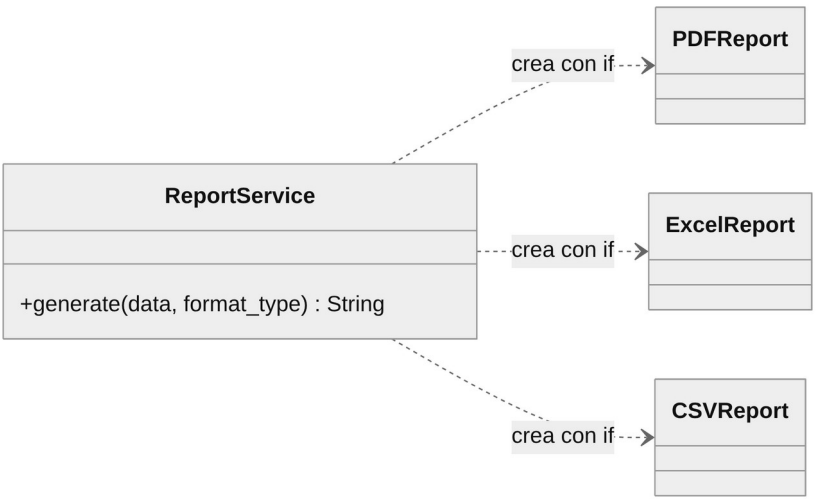
Apellido y nombre:	Figueroa, Ignacio – DNI 45.406.120
Comisión:	C4 - A26
Fecha de entrega:	29/10/2026
Lenguaje:	Python 3 (sin librerías externas)

1. El problema

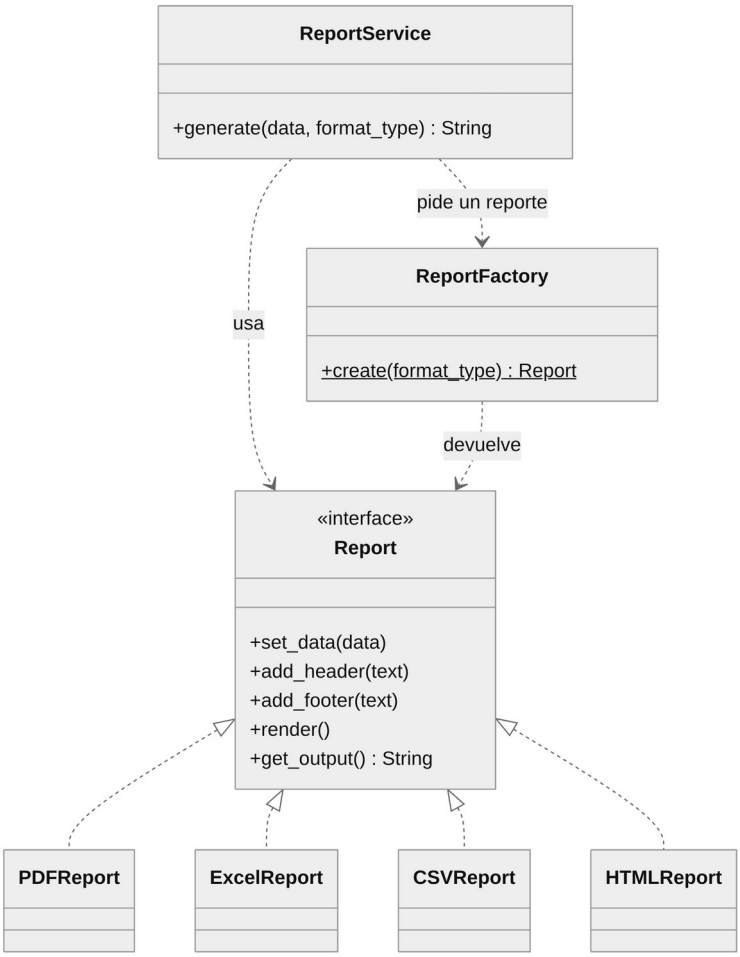
En el código original, ReportService hace dos cosas: decide qué clase de reporte crear (con un if por cada formato) y después lo usa. Por eso, cada vez que aparece un formato nuevo hay que modificar el servicio, aunque la forma de generar el reporte sea siempre la misma.

2. Diagramas UML

Antes



Después



La decisión de qué clase crear pasó a ReportFactory. ReportService solo conoce la interfaz Report y la fábrica, no las clases concretas.

3. Código antes

antes/report_service.py

```
# ANTES: código original (problemático)
# El servicio decide qué clase crear y además la usa.
from datetime import date


class PDFReport:
    def set_data(self, data): self.data = data
    def add_header(self, text): self.header = text
    def add_footer(self, text): self.footer = text
    def render(self): self.output = f"[PDF] {self.header} | {self.data} | {self.footer}"
    def get_output(self): return self.output


class ExcelReport:
    def set_data(self, data): self.data = data
    def add_header(self, text): self.header = text
    def add_footer(self, text): self.footer = text
    def render(self): self.output = f"[EXCEL] {self.header} | {self.data} | {self.footer}"
```

```
    def get_output(self): return self.output


class CSVReport:
    def set_data(self, data): self.data = data
    def add_header(self, text): self.header = text
    def add_footer(self, text): self.footer = text
    def render(self): self.output = f"{self.header}\n{self.data}\n{self.footer}"
    def get_output(self): return self.output


class ReportService:
    def generate(self, data, format_type):
        # decisión de construcción mezclada con la lógica de uso
        if format_type == "pdf":
            report = PDFReport()
        elif format_type == "excel":
            report = ExcelReport()
        elif format_type == "csv":
            report = CSVReport()
        else:
            raise ValueError("Formato no soportado")

        # lógica de uso, igual para todos
        report.set_data(data)
        report.add_header("Reporte Mensual")
        report.add_footer("Generado el " + date.today().strftime("%d/%m/%Y"))
        report.render()
        return report.get_output()


if __name__ == "__main__":
    print(ReportService().generate("ventas: 150", "pdf"))
    # Para agregar HTML habría que abrir ReportService y sumar otro elif.
```

4. Código después

despues/reports.py

```
# Producto y productos concretos
from abc import ABC, abstractmethod


class Report(ABC):
    """Interfaz común que cumplen todos los reportes."""

    @abstractmethod
    def set_data(self, data): ...

    @abstractmethod
    def add_header(self, text): ...

    @abstractmethod
    def add_footer(self, text): ...

    @abstractmethod
    def render(self): ...
```

```
    @abstractmethod
    def get_output(self) -> str: ...


class BaseReport(Report):
    """Guarda los datos comunes para no repetir lo mismo en cada formato."""

    def __init__(self):
        self.data = ""
        self.header = ""
        self.footer = ""
        self.output = ""

    def set_data(self, data):
        self.data = data

    def add_header(self, text):
        self.header = text

    def add_footer(self, text):
        self.footer = text

    def get_output(self):
        return self.output


class PDFReport(BaseReport):
    def render(self):
        self.output = f"[PDF] {self.header} | {self.data} | {self.footer}"


class ExcelReport(BaseReport):
    def render(self):
        self.output = f"[EXCEL] {self.header} | {self.data} | {self.footer}"


class CSVReport(BaseReport):
    def render(self):
        self.output = f"{self.header}\n{self.data}\n{self.footer}"


class HTMLReport(BaseReport):
    # Formato nuevo: se agregó sin tocar ReportService.
    def render(self):
        self.output = (f"<h1>{self.header}</h1>"
                       f"<p>{self.data}</p>"
                       f"<footer>{self.footer}</footer>")
```

despues/report_factory.py

```
# La fábrica concentra la decisión de qué clase crear.
#
# ¿Por qué Factory Method y no Abstract Factory?
# Abstract Factory sirve cuando hay que crear FAMILIAS de objetos que van juntos
# (por ejemplo botón + input + checkbox del mismo tema). Acá hay un solo tipo de
# objeto, Report, con distintas variantes. No hay familias, así que Abstract
```

```
# Factory agregaría clases sin necesidad.
#
# ¿Por qué Static Factory y no la variante GoF con jerarquía de creadores?
# Lo único que cambia entre formatos es la clase que se crea. Los pasos para
# generar el reporte son siempre los mismos. Con la versión GoF estricta tendría
# que hacer PdfReportService, ExcelReportService, etc. solo para cambiar una línea,
# y además el que llama tendría que elegir el servicio, lo que rompe la firma
# generate(data, format_type). Con una sola fábrica el cambio queda en un lugar.
from reports import Report, PDFReport, ExcelReport, CSVReport, HTMLReport


class ReportFactory:
    @staticmethod
    def create(format_type: str) -> Report:
        if format_type == "pdf":
            return PDFReport()
        if format_type == "excel":
            return ExcelReport()
        if format_type == "csv":
            return CSVReport()
        if format_type == "html":
            return HTMLReport()
        raise ValueError("Formato no soportado: " + format_type)
```

despues/report_service.py

```
# El servicio ya no sabe qué clase concreta usa: solo conoce la fábrica.
from datetime import date
from report_factory import ReportFactory


class ReportService:
    def generate(self, data, format_type):
        report = ReportFactory.create(format_type)
        report.set_data(data)
        report.add_header("Reporte Mensual")
        report.add_footer("Generado el " + date.today().strftime("%d/%m/%Y"))
        report.render()
        return report.get_output()
```

despues/main.py

```
from report_service import ReportService

servicio = ReportService()
datos = "ventas: 150"

for formato in ["pdf", "excel", "csv", "html"]:
    print(f"--- {formato} ---")
    print(servicio.generate(datos, formato))

print("--- formato inexistente ---")
try:
    servicio.generate(datos, "word")
except ValueError as e:
    print("Error:", e)
```



5. Ejecución

```
$ python main.py
```

```
--- pdf ---
[PDF] Reporte Mensual | ventas: 150 | Generado el 29/10/2026
--- excel ---
[EXCEL] Reporte Mensual | ventas: 150 | Generado el 29/10/2026
--- csv ---
Reporte Mensual
ventas: 150
Generado el 29/10/2026
--- html ---
<h1>Reporte Mensual</h1><p>ventas: 150</p><footer>Generado el 29/10/2026</footer>
--- formato inexistente ---
Error: Formato no soportado: word
```

HTMLReport funciona sin haber tocado ReportService. Para agregarlo solo cambiaron reports.py (la clase nueva) y ReportFactory (una línea más).

6. Justificación

El problema concreto era que ReportService tenía que cambiar cada vez que aparecía un formato nuevo, aunque su trabajo (cargar datos, poner encabezado y pie, renderizar) es siempre el mismo. Con Factory Method separé las dos responsabilidades: la fábrica decide qué clase crear y el servicio solo usa lo que recibe. Ahora ReportService no importa ninguna clase concreta y la firma generate(data, format_type) quedó igual. Agregar HTML solo implicó crear la clase y sumar una línea en la fábrica. Además, si un formato necesitara parámetros especiales para construirse, eso quedaría dentro de la fábrica y no ensuciaría el servicio.

7. Decisiones de diseño

¿Por qué Factory Method y no Abstract Factory?

Abstract Factory sirve para crear familias de objetos que tienen que ir juntos, como botón, input y checkbox de un mismo tema. Acá hay un solo tipo de objeto (Report) en distintas variantes, así que no hay familias. Usar Abstract Factory hubiera sumado clases sin aportar nada.

¿Por qué Static Factory y no la versión GoF con jerarquía de creadores?

Elegí la variante Static Factory. Entre un formato y otro solo cambia la clase que se crea; los pasos para generar el reporte son siempre los mismos. Con la versión GoF estricta tendría que crear PdfReportService, ExcelReportService, etc., cada una solo para cambiar una línea. Además, el que llama tendría que elegir qué servicio usar, y eso no respeta la firma generate(data, format_type) que pide la consigna. Con una sola fábrica el cambio queda en un único lugar. Si en el futuro cada formato necesitara pasos distintos al generar el reporte, ahí sí convendría pasar a la versión con jerarquía de creadores.